Furuno DRS4D-NXT

Echo Packet Encoding Guide

Bit-Level Protocol Analysis

J.C. Vaught

March 2026

Based on reverse-engineered protocol analysis of the Furuno DRS4D-NXT solid-state marine radar.

Contents

1 Echo Packet Overview **2**

2 Header **2**

3 Metadata (Bytes 8–11) **3**

3.1 Byte 8: Timestamp . . . 3

3.2 Byte 9: Sweep Count . . . 4

3.3 Byte 10: Sweep Length (Low Byte) . . . 4

3.4 Byte 11: Sweep Length (High Bits) and Encoding . . . 4

3.5 Worked Example . . . 5

4 Spoke Data **5**

4.1 Angle Sub-Header . . . 5

4.2 Echo Data . . . 6

4.3 Previous Spoke Buffer . . . 6

4.4 Display Conversion . . . 6

5 Echo Encoding **7**

5.1 Encoding 0: No Compression . . . 7

5.2 Encoding 1: Repeating . . . 8

5.2.1 Worked Example . . . 8

5.3 Encoding 2: Copying from Previous Spoke . . . 9

5.3.1 Worked Example . . . 9

5.4 Encoding 3: Combined Repeating and Copying . . . 10

5.4.1 Worked Example . . . 11

5.5 Encoding Comparison . . . 11

6 Doppler Mode **12**

6.1 Echo Byte Layout . . . 12

6.2 Status Field . . . 12

6.3 Interaction with Encoding . . . 13

6.4 Effective Bit Depths . . . 13

1 Echo Packet Overview

The Furuno DRS4D-NXT is a solid-state marine radar that streams echo data over a wired Ethernet connection. The radar transmits echo data as UDP datagrams on multicast group `239.255.0.2`, port `10024`. Each datagram constitutes one echo packet.

An echo packet contains a fixed 16-byte header followed by one or more spokes of data. A spoke is a single radial line of echo measurements, representing what the radar detects at a specific azimuth angle. As the antenna rotates, the radar produces spokes at incremental angles, packing several consecutive spokes into each UDP packet.

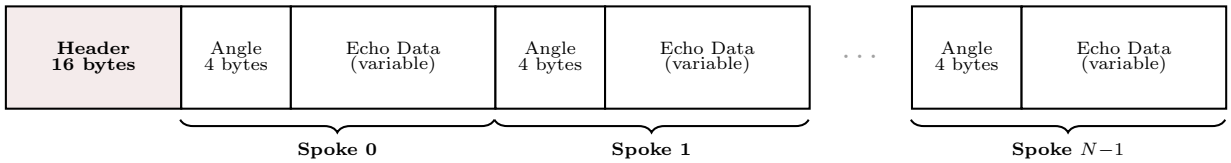


Figure 1: Echo packet structure. A 16-byte header is followed by one or more spokes, each containing an angle sub-header and variable-length echo data.

The radar completes 8,192 spokes per full 360 rotation. A single packet does not contain an entire rotation. Instead, spokes arrive across many packets over the course of one antenna revolution. The host software detects that a full rotation has completed when the azimuth value of a newly received spoke drops by more than 1,000 ticks compared to the previous spoke, indicating a wrap-around past zero.

After decoding, every spoke produces a fixed-length array of range bins. Each range bin represents the echo intensity at a specific distance from the radar along that spoke's azimuth. The number of range bins per spoke is typically 868, though this value varies with the radar's range setting and is specified in the packet header.

2 Header

Every echo packet begins with a 16-byte header. The header contains packet sequencing information, a timestamp, and metadata fields that the decoder needs to interpret the spoke data that follows.

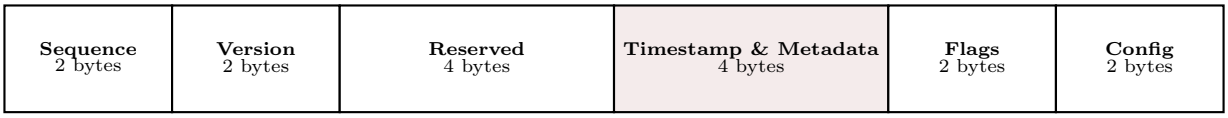


Figure 2: The six fields of the 16-byte echo packet header.

All multi-byte fields in the header use big-endian byte ordering. This is in contrast to the spoke sub-headers, which use little-endian ordering (see Section 4).

Table 1: Echo packet header fields.

Offset	Size	Field	Description
0–1	2 bytes	Sequence	Packet counter (u16 BE). Increments with each packet. Used for ordering and detecting packet loss.
2–3	2 bytes	Version	Protocol version (u16 BE). Always `0x0001`.
4–7	4 bytes	Reserved	Always `0x00000000`. Unused padding.
8–11	4 bytes	Timestamp & Metadata	Radar clock timestamp (u32 BE). Also contains three metadata fields packed at the bit level (see Section 3).
12–13	2 bytes	Flags	Always `0x1500`. Purpose not fully determined.
14–15	2 bytes	Config	Always `0xF009`. Purpose not fully determined.

Most header fields are static across all observed packets. The Sequence and Timestamp fields are the only values that change per packet. The Version, Reserved, Flags, and Config fields have been observed at fixed values across all range settings, scan speeds, and operating modes.

The minimum valid packet size is 20 bytes: the 16-byte header plus at least one 4-byte spoke sub-header. The critical information for decoding echo data is contained in bytes 8–11, described in the next section.

3 Metadata (Bytes 8–11)

Bytes 8 through 11 of the header serve a dual purpose. The echo header parser reads them as a single 32-bit big-endian timestamp representing the radar's internal clock value. Simultaneously, the metadata parser inspects bytes 9, 10, and 11 individually to extract three fields that control how the spoke data is decoded.

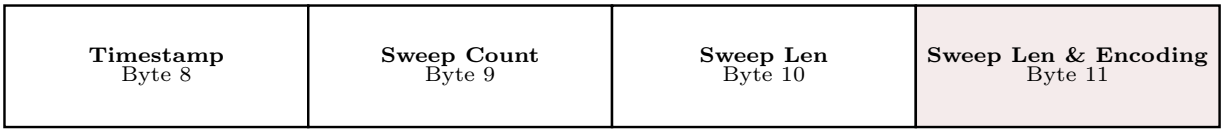


Figure 3: The four bytes of the timestamp and metadata region. Each byte carries a specific metadata field.

3.1 Byte 8: Timestamp

Byte 8 is the high byte of the 32-bit radar clock timestamp. It contains no metadata fields. Its value changes with each packet as the radar clock advances.

3.2 Byte 9: Sweep Count

Byte 9 encodes the sweep count, which specifies how many spokes are packed into this packet. The sweep count occupies the upper 7 bits of the byte (bits 7 through 1). Bit 0 is unused.

$$\texttt{sweep_count} = \texttt{data[9]} \gg 1 \tag{1}$$

The sweep count can range from 0 to 127, though typical values vary with the radar's scan speed and range setting. The decoder uses this value to know how many spoke sub-headers and echo data blocks to expect after the header.

3.3 Byte 10: Sweep Length (Low Byte)

Byte 10 contains the low 8 bits of the sweep length. The sweep length specifies how many range bins each spoke will contain after decoding. This is the first half of an 11-bit value; the remaining 3 bits are in byte 11.

3.4 Byte 11: Sweep Length (High Bits) and Encoding

Byte 11 contains three fields packed into a single byte. This is the most densely packed byte in the header.

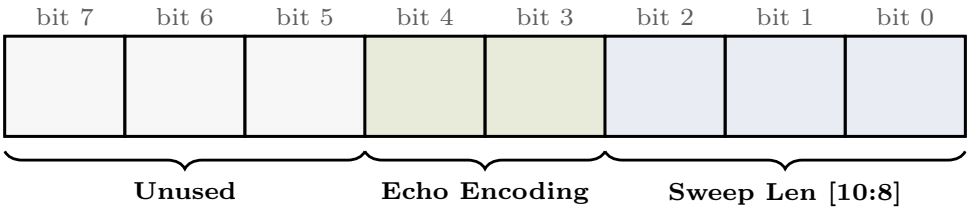


Figure 4: Bit-level layout of byte 11, showing three packed fields.

The three fields are extracted as follows.

Bits 7:5 (top 3 bits) are unused. They have been observed as zero across all captured packets.

Bits 4:3 (middle 2 bits) specify the echo encoding type. This 2-bit field selects one of four compression schemes that the radar has applied to the echo data in this packet. The four encoding types are described in detail in Section 5.

$$\texttt{encoding} = (\texttt{data[11]} \,\&\, \texttt{0x18}) \gg 3 \tag{2}$$

Bits 2:0 (bottom 3 bits) are the high 3 bits of the sweep length. Combined with byte 10, they form the complete 11-bit sweep length value.

$$\texttt{sweep_len} = ((\texttt{data[11]} \,\&\, \texttt{0x07}) \ll 8) \mid \texttt{data[10]} \tag{3}$$

The sweep length is an 11-bit unsigned integer with a theoretical range of 0 to 2,047. The typical observed value is 868 range bins, though this varies with the radar's range setting.

3.5 Worked Example

Consider bytes 8–11 with values `[0xAB, 0x0A, 0x64, 0x0B]`.

Byte 9 is `0x0A`. The sweep count is $\texttt{0x0A} \gg 1 = 5$, so this packet contains 5 spokes.

Byte 10 is `0x64`, which is 100 in decimal. This is the low byte of the sweep length.

Byte 11 is `0x0B` (`0b00001011`). The encoding field is $(\texttt{0x0B} \,\&\, \texttt{0x18}) \gg 3 = (\texttt{0x08}) \gg 3 = 1$, selecting Encoding 1 (repeating). The sweep length high bits are $\texttt{0x0B} \,\&\, \texttt{0x07} = 3$. The full sweep length is $(3 \ll 8) \mid 100 = 768 + 100 = 868$ range bins.

This packet therefore contains 5 spokes, each decoding to 868 range bins, compressed using Encoding 1.

4 Spoke Data

After the 16-byte header, the packet contains a sequence of spokes. The number of spokes is given by the sweep count field (Section 3). Each spoke consists of a fixed 4-byte angle sub-header followed by a variable-length block of encoded echo data.



Figure 5: Structure of a single spoke: a 4-byte angle sub-header followed by variable-length echo data.

4.1 Angle Sub-Header

The first 4 bytes of each spoke encode two angular measurements. Unlike the packet header, these fields use little-endian byte ordering.

Table 2: Spoke angle sub-header fields.

Offset	Size	Field	Description
0–1	2 bytes	Azimuth	Antenna rotational position (u16 LE). Measured in ticks.
2–3	2 bytes	Heading	Compass bearing (u16 LE). Measured in ticks.

Both fields use the same quantization: 8,192 ticks per full 360 rotation. This gives an angular resolution of $360/8{,}192 \approx 0.0439$ per tick. The conversion from ticks to degrees is:

$$\theta_{\text{deg}} = \frac{\texttt{ticks}}{8{,}192} \times 360 \tag{4}$$

The azimuth field tracks the antenna's rotational position as it sweeps around. Azimuth values increment with each successive spoke and wrap from approximately 8,191 back toward 0 at the start of each new rotation.

The heading field independently tracks the compass bearing from the radar's internal compass sensor. This value is updated per spoke and allows the host to convert radar-relative azimuth angles to true geographic bearings.

4.2 Echo Data

The echo data block follows the 4-byte angle sub-header and contains the encoded echo amplitudes for this spoke. The length of this block on the wire is variable — it depends on the encoding type and the data content. A spoke with many repeated values compresses to very few bytes, while a spoke with highly varied data may use nearly as many wire bytes as there are range bins.

After decoding, every spoke produces exactly `sweep_len` range bins, regardless of how many wire bytes were consumed. Each decoded range bin is a single byte representing echo intensity.

After each spoke's echo data is consumed, the byte count is padded to the next 4-byte boundary:

$$\texttt{consumed} = (\texttt{n} + 3)\ \&\ \neg 3 \tag{5}$$

This alignment ensures that each subsequent spoke's angle sub-header begins on a clean 4-byte boundary within the packet.

4.3 Previous Spoke Buffer

The decoder maintains a buffer called `prev_spoke` that stores the most recently decoded spoke's raw (pre-display-conversion) values. This buffer is updated after every spoke and persists across both spokes within the same packet and across consecutive UDP packets. The previous spoke buffer is essential for Encoding 2 and Encoding 3, which reference the previous spoke's range bins during decoding (see Sections 5.3 and 5.4).

4.4 Display Conversion

After decoding, every wire value undergoes a right-shift by 2 bits:

$$\texttt{display} = \texttt{wire} \gg 2 \tag{6}$$

This converts 8-bit wire values (0–252) to 6-bit display values (0–63). The shift is applied uniformly regardless of encoding type. As discussed in Section 5, this shift has the convenient property of stripping the encoding control bits from the low bits of raw data values, producing a clean 0–63 intensity range for all four encodings.

5 Echo Encoding

The radar applies one of four compression schemes to the echo data in each packet. The encoding type is specified by the 2-bit field in byte 11 of the header (see Section 3). All spokes within a single packet use the same encoding, but different packets within the same rotation may use different encodings.

The radar selects the encoding type adaptively based on the characteristics of the echo data at the time of transmission. The selection algorithm is implemented in the radar's firmware and has not been documented. In practice, Encoding 0 appears during sparse conditions, Encoding 1 during moderate spatial redundancy, and Encodings 2 and 3 during conditions where consecutive spokes share significant overlap.

All four encodings produce the same output: a fixed-length array of `sweep_len` range bins. After decoding, the $\gg 2$ display conversion is applied uniformly. The control bits used by Encodings 1, 2, and 3 occupy the lowest 1 or 2 bits of each byte, which are exactly the bits discarded by the display shift. This is almost certainly by design.

5.1 Encoding 0: No Compression

Encoding 0 is the simplest scheme. Every byte in the echo data maps directly to one range bin with no compression.

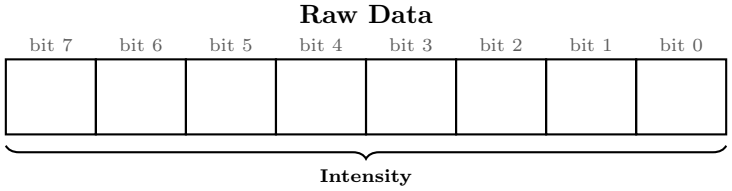


Figure 6: Encoding 0 byte layout. All 8 bits carry intensity data.

The first `sweep_len` bytes of the echo data are copied directly to form the decoded spoke. If fewer bytes are available than `sweep_len`, the remaining bins are zero-padded. The consumed byte count is rounded up to the next 4-byte boundary.

Wire values span the full 0–252 range with no restrictions. After the $\gg 2$ display shift, this produces 64 distinct display levels (0–63). Encoding 0 provides no compression benefit but guarantees exact representation of every range bin.

5.2 Encoding 1: Repeating

Encoding 1 introduces run-length compression using a single control bit. Bit 0 (the least significant bit) of each byte determines whether the byte carries raw intensity data or a repeat instruction.

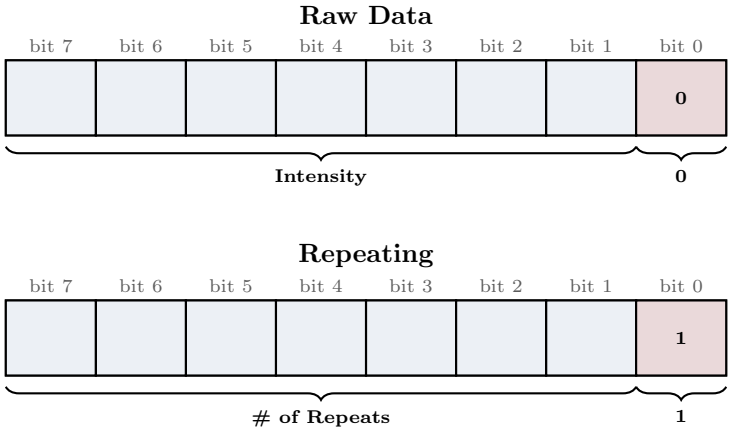


Figure 7: Encoding 1 byte layout. Bit 0 selects between raw data and repeating.

When bit 0 is 0, the byte is a raw intensity value. The full byte is stored as the echo amplitude for the current range bin, and the decoder remembers this value as the "current strength." Because bit 0 is always 0 for raw data bytes, the effective intensity values are restricted to even numbers: 0, 2, 4, ..., 252. This provides 127 distinct non-zero levels.

When bit 0 is 1, the byte is a repeat instruction. The decoder repeats the current strength value for a number of consecutive range bins. The repeat count is extracted from the upper 7 bits: `count` = `byte` $\gg 1$. A count value of 0 is treated as 128, allowing a single byte to fill up to 128 bins. The maximum explicitly encoded count is 127 (wire byte `0xFF`).

The current strength always refers to the most recently decoded raw data value, regardless of how many bins have been filled since it was set. Repeat instructions reference only this single value, making Encoding 1 a purely spatial compression scheme — it exploits runs of identical intensity values along the spoke.

5.2.1 Worked Example

Consider the wire byte sequence `[0x3C, 0x07, 0x50, 0x05, 0x00]`.

`0x3C` has bit 0 = 0, so it is raw data with intensity 60. The current strength becomes 60. Bin 0 = 60.

`0x07` has bit 0 = 1, so it is a repeat. Count = `0x07` $\gg 1 = 3$. The current strength (60) is written into bins 1, 2, and 3.

`0x50` has bit 0 = 0, raw data with intensity 80. The current strength becomes 80. Bin 4 = 80.

`0x05` has bit 0 = 1, repeat. Count = `0x05` $\gg 1 = 2$. Bins 5 and 6 = 80.

`0x00` has bit 0 = 0, raw data with intensity 0. Bin 7 = 0.

The decoded spoke is `[60, 60, 60, 60, 80, 80, 80, 0]`, produced from 5 wire bytes — a compression ratio of 5:8 in this example.

5.3 Encoding 2: Copying from Previous Spoke

Encoding 2 uses the same 1-bit control flag as Encoding 1, but the meaning of the repeat instruction changes. Instead of repeating the current strength, the decoder copies values from the previously decoded spoke at the corresponding range bin positions.

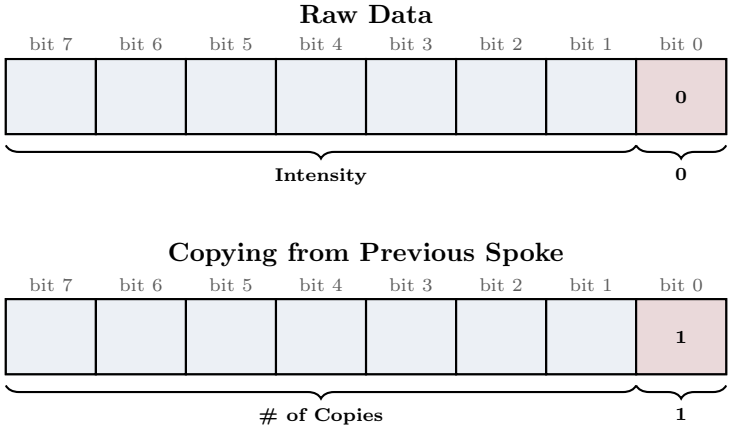


Figure 8: Encoding 2 byte layout. Bit 0 selects between raw data and copying from the previous spoke.

When bit 0 is 0, the byte is raw data, identical to Encoding 1.

When bit 0 is 1, the decoder copies a number of consecutive range bins from the previous spoke. The copy starts at the current output index and reads from the same index in the previous spoke's data. The count formula is the same as Encoding 1: `count` = `byte` $\gg 1$, with 0 meaning 128.

This scheme exploits temporal redundancy. Consecutive radar spokes often observe similar echo patterns, particularly for stationary targets, coastlines, and consistent sea clutter. By encoding only the values that differ from the previous spoke, the radar can achieve significant compression for slowly-changing scenes.

The first spoke in a packet (`sweep_idx == 0`) cannot use previous-spoke references because no prior spoke has been decoded in the current packet context. The decoder handles this by falling back to Encoding 1 behavior (repeating the current strength) for the first spoke, then using full Encoding 2 for subsequent spokes. The `prev_spoke` buffer persists across UDP packets, so the temporal reference chain is maintained across packet boundaries.

5.3.1 Worked Example

Suppose the previous spoke contains `[60, 60, 80, 80, 40, 40, 20, 20]` and the wire bytes for the current spoke are `[0x50, 0x05, 0x3C, 0x07]`.

`0x50` has bit 0 = 0, raw data with intensity 80. Bin 0 = 80.

`0x05` has bit 0 = 1, copy from previous spoke. Count = `0x05` ≫ 1 = 2. Copy bins 1 and 2 from the previous spoke: bin 1 = 60, bin 2 = 80.

`0x3C` has bit 0 = 0, raw data with intensity 60. Bin 3 = 60.

`0x07` has bit 0 = 1, copy from previous spoke. Count = `0x07` ≫ 1 = 3. Copy bins 4, 5, and 6 from the previous spoke: bin 4 = 40, bin 5 = 40, bin 6 = 20.

The decoded spoke is `[80, 60, 80, 60, 40, 40, 20]`, mixing new raw values with temporally-referenced copies from the previous rotation.

5.4 Encoding 3: Combined Repeating and Copying

Encoding 3 is the most sophisticated scheme. It uses two control bits (bits 1:0) instead of one, providing three distinct operations within a single encoding. This combines the spatial compression of Encoding 1 with the temporal compression of Encoding 2.

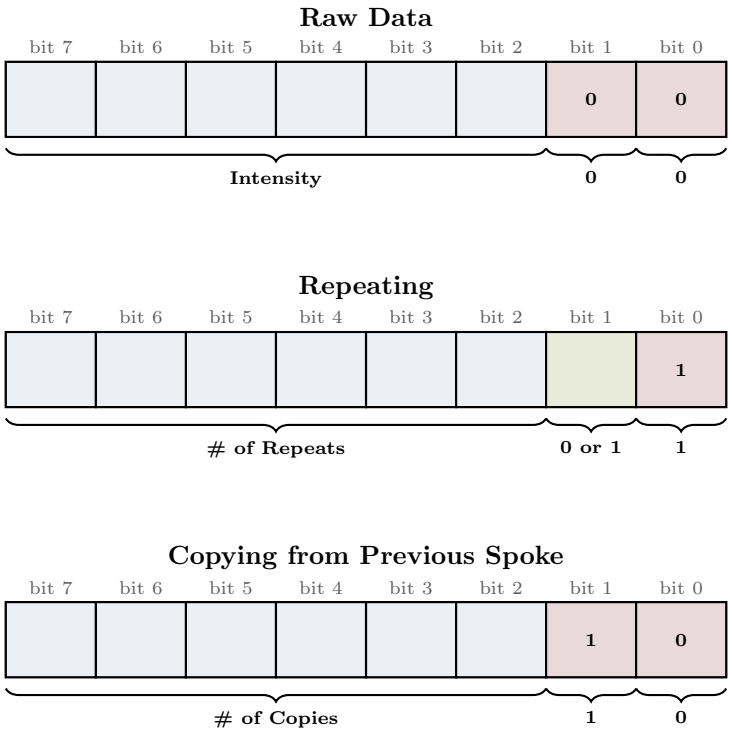


Figure 9: Encoding 3 byte layout. Bits 1:0 select between raw data, repeating, and copying from the previous spoke.

When bits 1:0 are `00`, the byte is raw data. Since the bottom 2 bits are always zero for raw data bytes, the effective intensity values are restricted to multiples of 4: 0, 4, 8, ..., 252. This provides 64 distinct intensity levels, which maps exactly to the 0–63 display range after the ≫ 2 conversion.

When bit 0 is 1 (bits 1:0 are `01` or `11`), the decoder repeats the current strength value, identical to Encoding 1. The count is extracted from the upper 6 bits: `count` = `byte` ≫ 2. A count of 0 means 64. The maximum explicit count is 63.

When bits 1:0 are `10`, the decoder copies from the previous spoke at the corresponding range bin positions, identical to Encoding 2. The count formula is the same: `count = byte` $\gg 2$, with 0 meaning 64. During a copy operation, the internal strength variable is updated to the last copied value, so a subsequent repeat instruction will use the copied value rather than the previous raw data value.

The trade-off for combining both modes is reduced resolution. Only 6 bits remain for the count field (maximum 63, vs 127 in Encodings 1 and 2), and raw data values lose their bottom 2 bits (64 distinct levels, vs 127 in Encodings 1 and 2). However, the ability to mix spatial and temporal references within a single spoke provides the highest compression ratios for mixed scenes.

5.4.1 Worked Example

Suppose the previous spoke contains `[40, 40, 80, 80, 60, 60]` and the wire bytes are `[0x3C, 0x09, 0x0A, 0x50]`.

`0x3C` has bits 1:0 = `00`, raw data. Intensity = `0x3C` = 60. The current strength becomes 60. Bin 0 = 60.

`0x09` has bits 1:0 = `01`, bit 0 is 1, so this is a repeat. Count = `0x09` $\gg 2 = 2$. The current strength (60) is written into bins 1 and 2.

`0x0A` has bits 1:0 = `10`, copy from previous spoke. Count = `0x0A` $\gg 2 = 2$. Copy bins 3 and 4 from the previous spoke: bin 3 = 80, bin 4 = 60.

`0x50` has bits 1:0 = `00`, raw data. Intensity = `0x50` = 80. Bin 5 = 80.

The decoded spoke is `[60, 60, 60, 80, 60, 80]`, demonstrating all three operations in 4 wire bytes producing 6 range bins.

5.5 Encoding Comparison

Table 3: Comparison of the four echo encoding schemes.

Property	**Enc 0**	**Enc 1**	**Enc 2**	**Enc 3**
Control bits	0	1 (bit 0)	1 (bit 0)	2 (bits 1:0)
Intensity bits	8	7	7	6
Raw data values	0–252	even only	even only	multiples of 4
Distinct levels	127	127	127	64
Repeat last value	—	yes	—	yes
Copy from prev spoke	—	—	yes	yes
Max count	—	128	128	64
Compression type	none	spatial	temporal	both

The $\gg 2$ display shift applied after decoding discards the bottom 2 bits of every wire value. This means the intensity restrictions imposed by the encoding control bits (even-only for Encodings 1 and 2, multiples-of-4 for Encoding 3) have no effect on the final display values. All four encodings produce the same 64-level (0–63) display range.

6 Doppler Mode

The radar supports an optional Doppler target analysis mode that repurposes the most significant bit of each echo byte to indicate whether a target is moving. Doppler mode is activated via the `$SEF` command (register `0xEF`) and can be set to one of three states: off (0), target analyzer (1), or rain mode (2).

6.1 Echo Byte Layout

When Doppler mode is active, the interpretation of each echo byte changes. In normal mode, all 8 bits encode intensity. In Doppler mode, bit 7 becomes a motion flag and the remaining 7 bits encode a reduced-range intensity value.

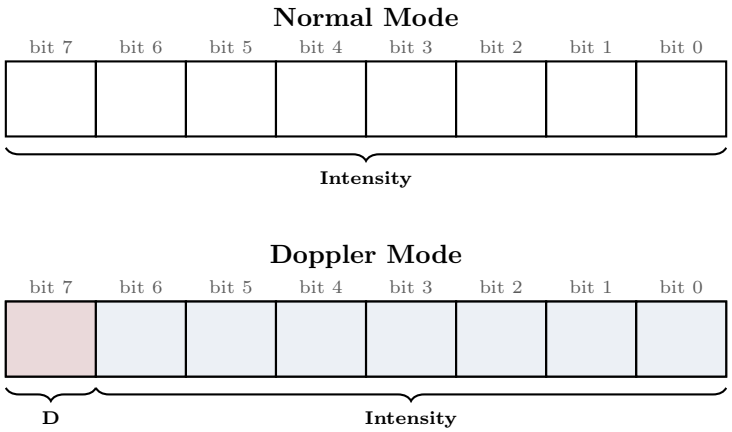


Figure 10: Echo byte layout in normal mode vs Doppler mode.

In normal mode, the full byte represents echo intensity spanning 0 to 252. In Doppler mode, bit 7 serves as the D flag: 0 indicates a stationary target and 1 indicates a moving target. Bits 6 through 0 encode intensity compressed to 7 bits, spanning 0 to 126. The extraction operations are:

$$D = \texttt{byte} \,\&\, \texttt{0x80} \tag{7}$$

$$\texttt{intensity} = \texttt{byte} \,\&\, \texttt{0x7F} \tag{8}$$

For example, the wire byte `0xC7` in Doppler mode decodes to $D = 1$ (moving target) with intensity `0x47` $= 71$.

6.2 Status Field

The Doppler mode status is communicated per spoke through a status field. Bit 8 of the status field (`0x100`) indicates whether Doppler data is present in the echo bytes. Bits 2:0 independently encode the pulse type.

Table 4: Observed status field values.

Status	Binary	Meaning
1 (0x01)	0_0000_0001	Normal echo, long pulse
4 (0x04)	0_0000_0100	Normal echo, short pulse
257 (0x101)	1_0000_0001	Doppler echo, long pulse
260 (0x104)	1_0000_0100	Doppler echo, short pulse

6.3 Interaction with Encoding

Doppler packing is applied by the radar hardware before the echo data is compressed. The encoding schemes described in Section 5 operate on the raw 8-bit bytes without any knowledge of whether bit 7 carries a D flag or intensity data. The encoding control bits (bit 0 for Encodings 1 and 2, bits 1:0 for Encoding 3) occupy the low end of the byte and do not interfere with the D flag in bit 7.

This means the D flag passes through the entire encode/decode pipeline untouched. However, the current software implementation does not extract the D flag after decoding. The uniform $\gg 2$ display shift causes the D flag to bleed from bit 7 into bit 5 of the display value, adding 32 to the display intensity of moving targets. This produces a visual effect where moving targets appear brighter, but it is not an intentional Doppler visualization.

6.4 Effective Bit Depths

The combination of Doppler mode and encoding type determines the effective number of intensity bits available.

Table 5: Effective intensity resolution across modes and encodings.

Mode	Encoding	Wire Intensity Bits	Display Levels
Normal	0	8	64
Normal	1 / 2	7	64
Normal	3	6	64
Doppler	0	7	32 (+32 if moving)
Doppler	1 / 2	6	32 (+32 if moving)
Doppler	3	5	32 (+32 if moving)

In Doppler mode with Encoding 3, only 5 bits of intensity information remain in the wire byte. After the display shift, this yields 32 distinct intensity levels for stationary targets and a separate 32 levels (offset by +32) for moving targets.